

2. Scale to 500k+ records

Partition work by source, query and date ranges; keep each unit independently restartable.

Challenge	Implementation	Key features
Partitioning	Source/query/date-range work units	Parallelizes acquisition without replaying completed ranges.
Bounded workers	asyncio/aiohttp + connection pools + queue limits	Prevents unbounded memory, sockets and downstream pressure.
Checkpoints	PostgreSQL acquisition checkpoints	Restarts from the last durable offset after interruption.
Idempotency	Canonical URL + SHA-256 content hash	Makes duplicate delivery safe across retries and workers.
Coordination	Redis queues/cache/rate limiting	Coordinates distributed work while PostgreSQL remains the system of record.

3. 413 / 429 resilience

Provider and payload failures are isolated so one transient limit does not stop the pipeline.

Failure	Response	Outcome
HTTP 413	Reduce token budget and recursively split the affected chunk.	Large documents continue without losing the record.
HTTP 429	Honor Retry-After when present; otherwise use exponential backoff + jitter with bounded retries.	Smooths bursts and avoids synchronized retry storms.
Provider failure	Gemini Flash → Groq → DeepSeek.	Provider isolation preserves throughput when one model is unavailable.
Validation failure	Validate structured output with Pydantic before persistence.	Rejects malformed or non-conforming records.

4. Freshness & distributed deduplication

URL identity	Content identity	Freshness gate	Date evidence	Processing state
Canonicalize URLs and enforce database uniqueness.	SHA-256 hash detects repeated content.	News/jobs require normalized publication time within the previous 24 hours.	Meta tags, JSON-LD and relative timestamps are parsed; ambiguous dates are rejected.	Collection and processing timestamps support incremental refresh.

5. Storage architecture

Keep the primary model simple: structured, provenance-heavy records belong in a transactional system of record.

Layer	Source	Notes
Primary	PostgreSQL	Raw documents, checkpoints, canonical entities, mappings, transactional uniqueness and deterministic joins.
Coordination	Redis	Work coordination, caching and distributed rate limiting.
Optional	Vector / graph database	Introduce only when a demonstrated semantic-search or relationship-query requirement justifies the operational cost.
Output	Google Sheets	Six-tab review/export surface for Startups, Products, Research Papers, Jobs, News and Entity Mapping Log.

6. Anti-bot & JavaScript-heavy sites

The crawler is designed for compliant acquisition, not challenge circumvention.

1. PREFER	2. ESCALATE	3. RESPECT	4. STOP
Official feeds/APIs and ordinary HTTP first.	Use Playwright Async only for legitimately accessible JS-rendered pages.	Robots.txt, rate limits, terms and publisher controls.	No CAPTCHA, Cloudflare or Datadome bypass.

7. Observability & correctness

Provenance	Structured logging	Testability	Production CI
Every canonical record retains source URL, source name, collection time and content hash.	Expose source, batch, offset, retry count and provider for operational diagnosis.	Cover URL normalization, hashing, pagination, checkpoint upserts and live-source integration.	Use mocked HTTP fixtures for deterministic, repeatable tests.

Architecture outcome

A restartable and auditable ingestion system that scales through partitioned work, bounded concurrency and durable checkpoints; protects data quality with strict freshness, provenance and deterministic resolution; and protects model calls with adaptive chunking, rate-limit backoff and provider fallback.